EcoWay Pursuit: booking and payouts architecture review

Prepared for: session with Jason Jefferson (staff software engineer)
Company: EcoWay Pursuit, a marketplace for booking electric-vehicle charging bays at private sites

Our system today

Drivers book 30-minute charging slots at bays owned by hotels, offices and car parks. Site owners are paid out through Stripe Connect. The product is a Ruby on Rails monolith with one PostgreSQL database, a Sidekiq job queue and a React Native app. Four engineers work on it.

- About 40,000 bookings per month, with peaks on Friday afternoons
- Roughly 0.3% of peak-hour bookings end up double-booked: two drivers get the same bay and slot
- Payouts to 180 site owners run as one nightly batch job, which has failed or partly failed four times in the last two months
- Stripe webhooks are processed inline in a Rails controller

What is going wrong

Double bookings cost us refunds and angry drivers. Our current check reads the bay's bookings, then inserts a new one if the slot looks free; we suspect two requests pass the check at the same moment. When the payout job fails halfway we re-run it by hand, and we are not fully sure some owners have not been paid twice.

Questions for the session

- What is the most reliable way to stop double bookings: database constraints, row locks, or something else?
- Should payouts move out of the nightly batch into per-booking transfers or a separate service?
- How should we make webhook handling and payouts safe to retry?

Constraints

We have a board demo on November 30 and want the double-booking fix live before then. We would rather not split the monolith into many services this year.